26-1 이니로
알고리즘 멘토링

멘토 - 김수성

4주차 – Sparse Table

합성함수와 쿼리 /17435

백준 17435 / https://www.acmicpc.net/problem/17435

함수 f : {1, 2, ⋯, M} -> {1, 2, ⋯, M}가 있고,

f1(x) = f(x)
fn+1(x) = f(fn(x)) 라고 할 때

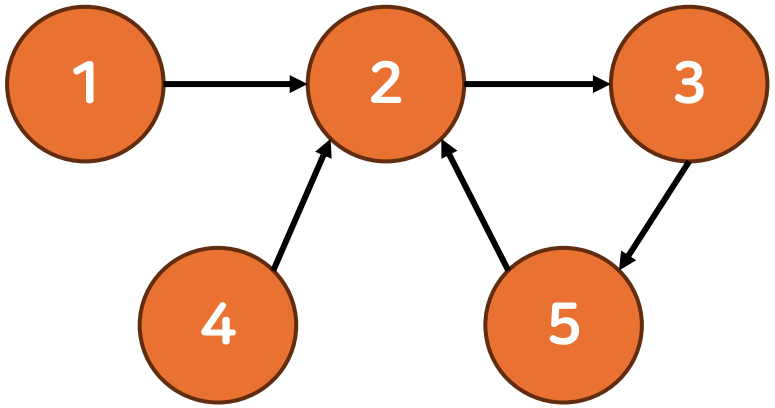
fn(x)를 구해야 함

합성함수와 쿼리 /17435

함수형 그래프

모든 정점이 outdegree를 1개만 가지는 그래프

간선이 N개, 항상 사이클이 존재함
사이클에 속하지 않는 정점은 간선을 따라가면
결국 사이클에 도달하게 됨

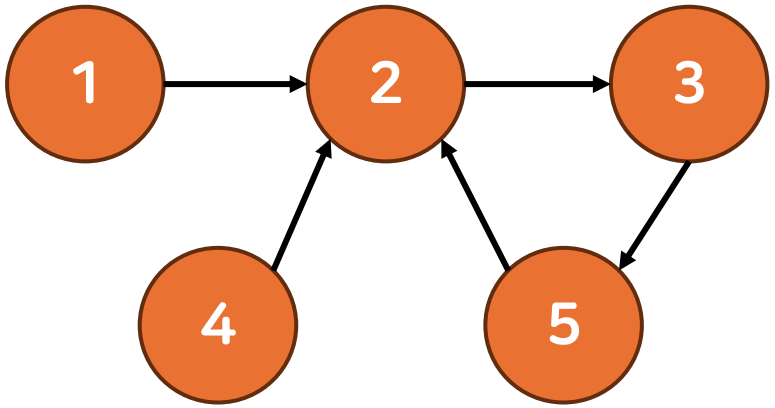


합성함수와 쿼리 /17435

함수형 그래프

f(x)를 한번 연산하는 것은 그래프 상에서
간선을 따라 이동하는 것과 동일함

f(1) = 2 / f(2) = 3 / f(3) = 5
f(4) = 2 / f(5) = 2



합성함수와 쿼리 /17435

따라서 fn(x)는 정점 x에서 간선을 따라서
n번 움직이면 구할 수 있음

쿼리 한번에 O(N)의 시간 복잡도가 들고
쿼리가 M개 있음 -> O(N * M)
N이 2e5, M이 5e5이므로 시간 초과가 남

합성함수와 쿼리 /17435

쿼리 한번에 O(N)의 시간 복잡도가 들고
쿼리가 M개 있음 -> O(N * M)
N이 2e5, M이 5e5이므로 시간 초과가 남

-> 여러 번의 연산을 압축하는 방법?

합성함수와 쿼리 /17435

여러 번의 연산을 압축하는 방법?
f10(x) = f4(f6(x))

즉 x에서 10번 이동하는 것과
x에서 4, 6번 이동하는 것은 같음
-> 연산을 작은 여러 연산들로 나눌 수 있음

합성함수와 쿼리 /17435

연산을 작은 여러 연산들로 나눌 수 있음

여러 연산들로 나눌 수 있으려면
나눈 것을 합칠 때의 순서에 영향을 받으면 안됨

-> 결합 법칙이 필요함 / op(op(a, b), c) = op(a, op(b, c))

합성함수와 쿼리 /17435

그럼 연산을 어떻게 나누는 게 좋을까?

모든 자연수는 이진수로 표현 할 수 있음
-> 모든 자연수는 2의 제곱의 합으로 표현 할 수 있음
ex) 11 = 0b00001011

합성함수와 쿼리 /17435

모든 자연수는 이진수로 표현 할 수 있음
-> 모든 자연수는 2의 제곱의 합으로 표현 할 수 있음
ex) 11 = 0b00001011

이진수로 바꾸면 켜진 비트의 합으로 표현 가능함
따라서 f11(x) = f8(f2(f1(x)))

합성함수와 쿼리 /17435

이진수로 바꾸면 켜진 비트의 합으로 표현 가능함
따라서 f11(x) = f8(f2(f1(x)))

2의 제곱 형태의 fn(x)를 저장하면
비트 수 만큼의 연산으로 fn(x)를 계산할 수 있음

-> 2의 제곱 형태의 fn(x)를 빨리 계산해야 함

합성함수와 쿼리 /17435

2의 제곱 형태의 fn(x)를 빨리 계산해야 함
lift[i][x] = 정점 x에서 2^i 만큼 이동 했을 때의 정점

우선 i = 0을 채우는 건 쉬움
lift[0][x] = 정점 x에서 2^0 = 1 만큼 이동 했을 때의 정점
= f(x)

```
cin >> n;
for(int i = 1;i <= n;i++) cin >> lift[0][i];
```

합성함수와 쿼리 /17435

lift[i][x] = 정점 x에서 2^i 만큼 이동 했을 때의 정점

i > 0을 채우는 건 결합 법칙의 성질로 채울 수 있음
lift[i][x] = lift[i – 1][lift[i – 1][x]]

정점 x에서 2^i 만큼 이동 한 것은
정점 x에서 2^(i – 1) 만큼 이동하고
추가로 2^(i – 1)만큼 이동한 것과 같음

```
void build(){
    for(int i = 1;i < 22;i++){
        for(int x = 1;x <= n;x++){
            int mid = lift[i - 1][x];
            lift[i][x] = lift[i - 1][mid];
        }
    }
}
```

합성함수와 쿼리 /17435

쿼리를 처리하는 법

이진수는 각 비트가 켜져 있거나 꺼져 있으므로
비트가 켜져 있을 때만 연산을 하면 됨

```
int query(int x, int v){
    for(int i = 0;i < 22;i++){
        if((v & (1 << i)) == 0) continue;
        x = lift[i][x];
    }

    return x;
}
```

합성함수와 쿼리 /17435

시간 복잡도
lift 전처리 -> O(N log N)
쿼리 -> O(M log N)

-> O((N + M) * log N)

```
int lift[22][MAX];

void build(){
    for(int i = 1;i < 22;i++){
        for(int x = 1;x <= n;x++){
            int mid = lift[i - 1][x];
            lift[i][x] = lift[i - 1][mid];
        }
    }
}

int query(int x, int v){
    for(int i = 0;i < 22;i++){
        if((v & (1 << i)) == 0) continue;
        x = lift[i][x];
    }

    return x;
}

void run(){
    cin >> n;
    for(int i = 1;i <= n;i++) cin >> lift[0][i];
    build();

    cin >> m;
    while(m--){
        int v, x; cin >> v >> x;
        cout << query(x, v) << "\n";
    }
}
```

질문?

최솟값 / 10868

백준 10868 / https://www.acmicpc.net/problem/10868

N개의 정수 배열 A와 M개의 쿼리가 주어짐

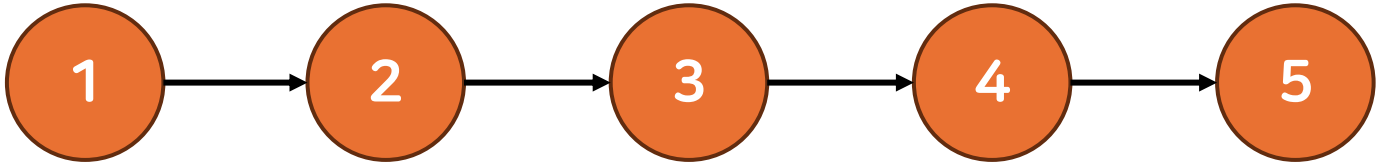
쿼리는 L, R 형태로 주어지고, 각 쿼리에 대해
배열 A의 L번째 수부터 R번째 수 중에서 제일 작은 수를 출력

최솟값 / 10868

우선 배열을 그래프로 모델링 해보자

이전 문제와 다르게 f(x)를 간선을 타는 이동으로 정의하면
최솟값과 무관한 그래프가 되어서 풀 수 없음

A = { 31, 20, 24, 11, 5 }

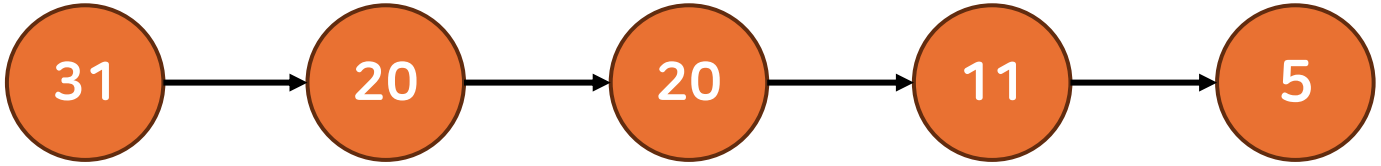


최솟값 / 10868

우선 배열을 그래프로 모델링 해보자

반대로 f(x)를 최솟값으로 정의하면
간선을 탈 수록 값이 줄어들고, 시작점의 정보를 알 수가 없음

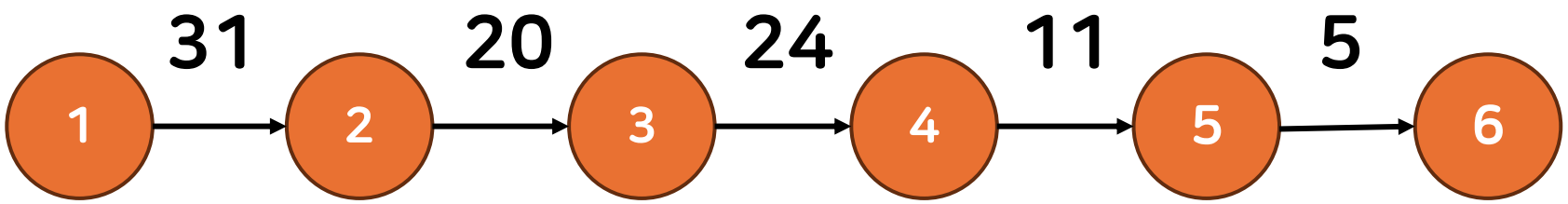
A = { 31, 20, 24, 11, 5 }



최솟값 / 10868

정점을 번호, 간선을 최솟값으로 만든 그래프를 생각 해보자

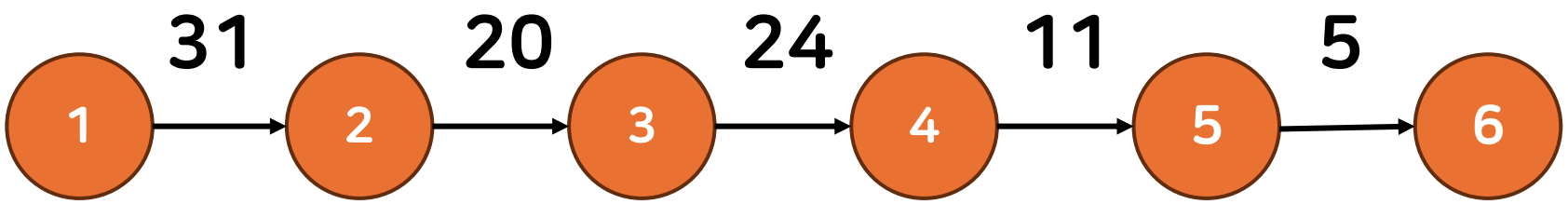
마지막 번호도 간선을 가져야 하므로 N + 1의 더미 정점을 추가
A = { 31, 20, 24, 11, 5 }



최솟값 / 10868

이전과 다르게 간선에도 값이 생겼으므로 lift 배열 이외에 cost 배열도 관리 해보자

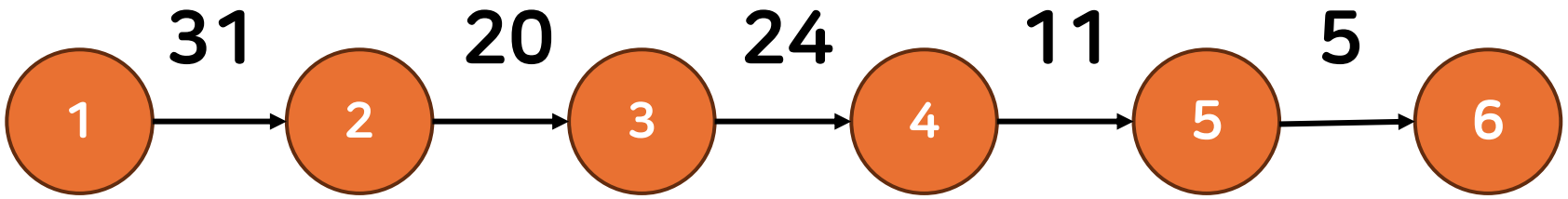
min 연산도 f(x)와 마찬가지로 결합 법칙이 성립하기 때문에 Sparse Table을 만들 수 있음



최솟값 / 10868

lift[0][1] = 2 / lift[0][2] = 3 / lift[0][3] = 4
lift[0][4] = 5 / lift[0][5] = 6 / lift[0][6] = 6
더미 정점의 lift 값을 자기 자신으로 해야 값이 틀어지지 않음

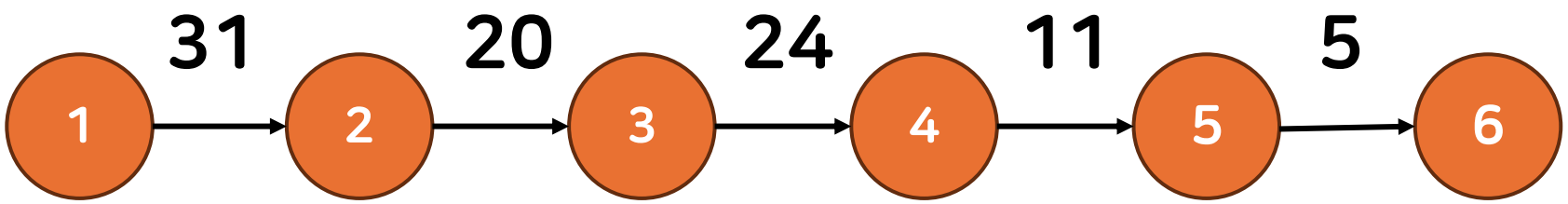
```
void build(){
    for(int i = 1;i < 22;i++){
        for(int x = 1;x <= n + 1;x++){
            int mid = lift[i - 1][x];
            lift[i][x] = lift[i - 1][mid];
        }
    }
}
```



최솟값 / 10868

cost[i][j] = i ~ i + 2^j – 1 구간의 최솟값

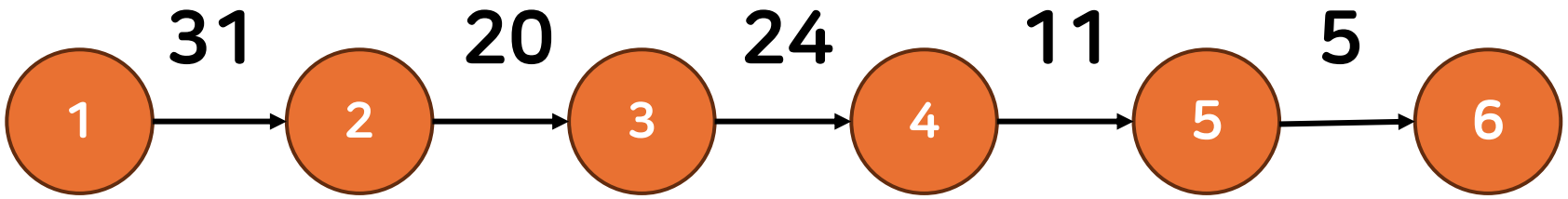
cost[0][1] = 31 / cost[0][2] = 20 / cost[0][3] = 24
cost[0][4] = 11 / cost[0][5] = 5 / cost[0][6] = INF
더미 정점의 cost 값을 큰 값으로 해야 정답에 영향을 주지 않음



최솟값 / 10868

cost[i][j] = i ~ i + 2^j – 1 구간의 최솟값
cost[i][j] = min(cost[i - 1][j], cost[i – 1][mid])

```
void build(){
    for(int i = 1;i < 22;i++){
        for(int x = 1;x <= n + 1;x++){
            int mid = lift[i - 1][x];
            lift[i][x] = lift[i - 1][mid];
            cost[i][x] = min(cost[i - 1][x], cost[i - 1][mid]);
        }
    }
}
```



최솟값 / 10868

시간 복잡도

build() -> O(N log N)

query() -> O(log N)

-> O((N + M) log N)

```
int lift[22][MAX], cost[22][MAX];
void build(){
    for(int i = 1;i < 22;i++){
        for(int x = 1;x <= n + 1;x++){
            int mid = lift[i - 1][x];
            lift[i][x] = lift[i - 1][mid];
            cost[i][x] = min(cost[i - 1][x], cost[i - 1][mid]);
        }
    }
}

int query(int x, int v){
    int ret = 0x3f3f3f3f;
    for(int i = 0;i < 22;i++){
        if((v & (1 << i)) == 0) continue;
        ret = min(ret, cost[i][x]);
        x = lift[i][x];
    }

    return ret;
}

void run(){
    cin >> n >> m;
    memset(cost, 0x3f, sizeof(cost));
    lift[0][n + 1] = n + 1;

    for(int i = 1;i <= n;i++) cin >> cost[0][i], lift[0][i] = i + 1;
    build();

    while(m--){
        int l, r; cin >> l >> r;
        cout << query(l, r - l + 1) << "\n";
    }
}
```

질문?

LCA2 / 11438

백준 11438 / https://www.acmicpc.net/problem/11438

N개의 정점을 가진 트리와 M개의 쿼리가 주어짐

쿼리는 A, B 형식으로 주어지고
각 쿼리에 대해서 lca(A, B)를 출력해야 함

LCA2 / 11438

우선 나이브하게 lca를 구하는 법을 생각해보자

lca = 최소 공통 조상
두 정점 a, b에 대해서 공통 조상 중 거리가 가장 가까운 정점

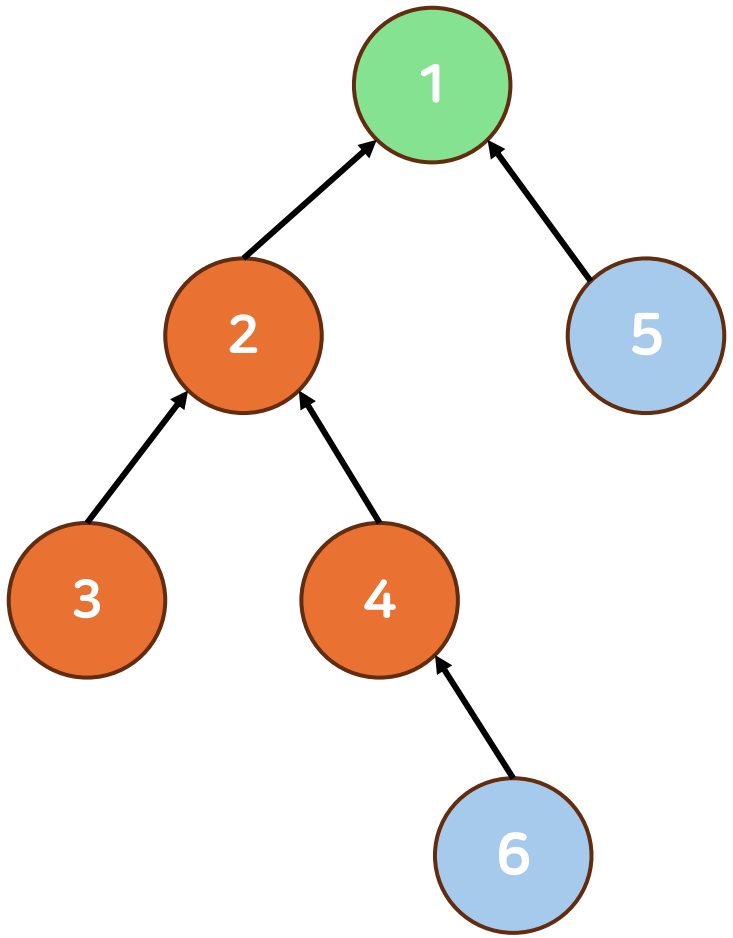
-> 정점 a, b의 조상을 타고 올라가면서
같은 정점이 되는 첫 번째 정점을 찾으면 됨

LCA2 / 11438

정점 a, b의 조상을 타고 올라가면서
같은 정점이 되는 첫 번째 정점을 찾으면 됨

하지만 a, b의 깊이가 다를 경우
lca에 도달하는 시간이 다름

-> 깊이를 먼저 맞추고 올라가면 됨

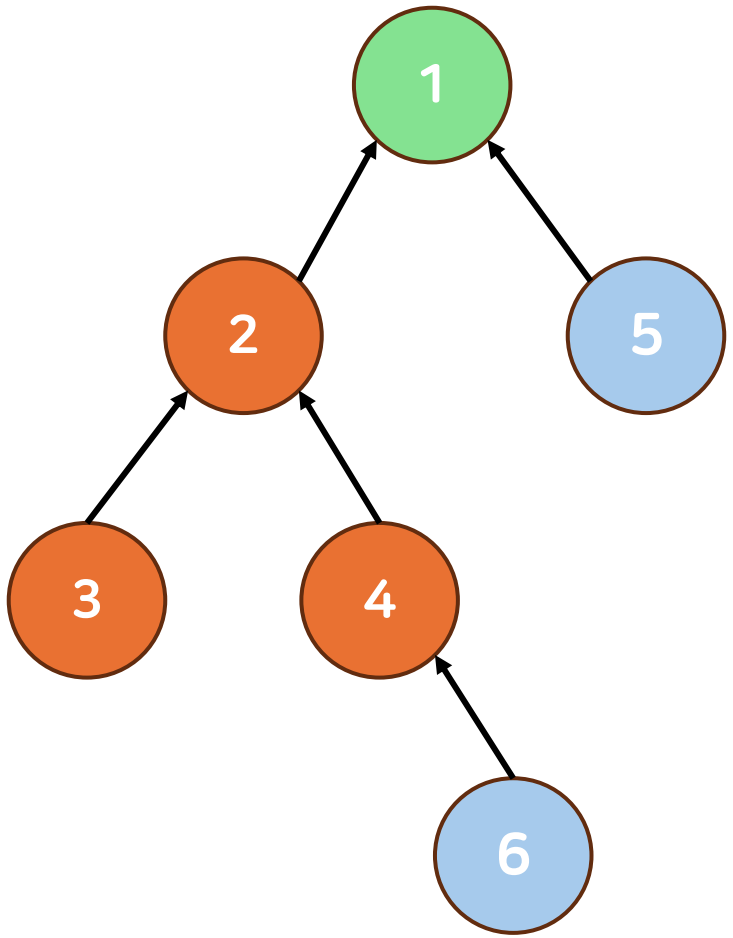


LCA2 / 11438

나이브하게 lca를 구하는 법

-> a, b의 깊이를 먼저 맞추고
조상을 타고 올라가면서 a, b가 같은
정점이 되면 그 정점이 lca

시간 복잡도
O(depth) -> O(N)

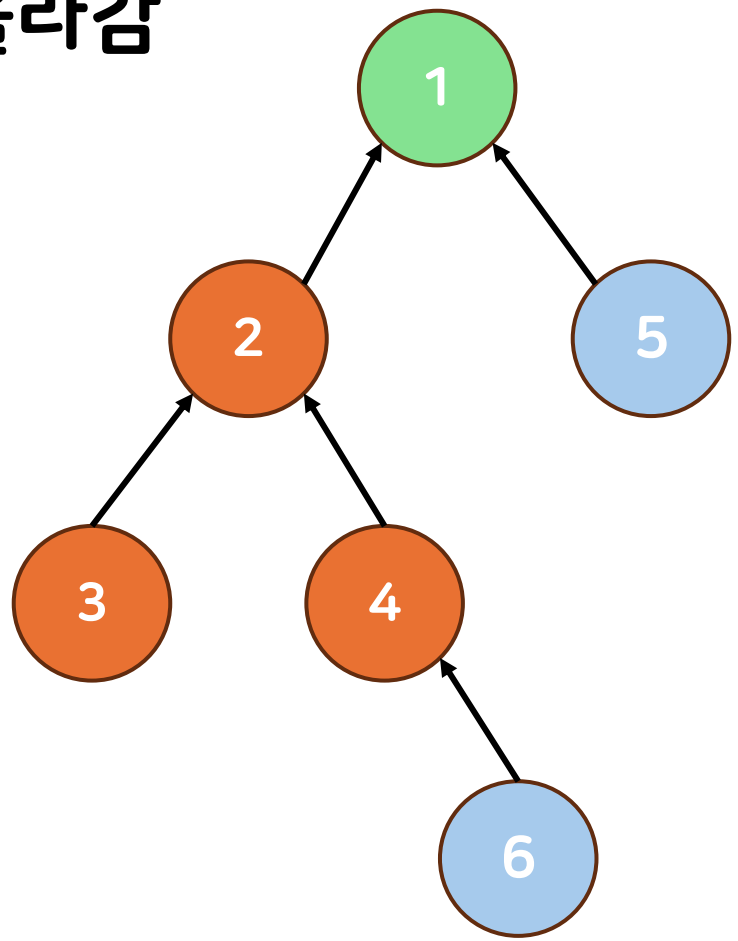


LCA2 / 11438

O(N)의 시간이 걸리는 이유

한번에 깊이가 1 차이 나는 조상으로 올라감

한번에 여러 번 올라가는 방법은 없나?

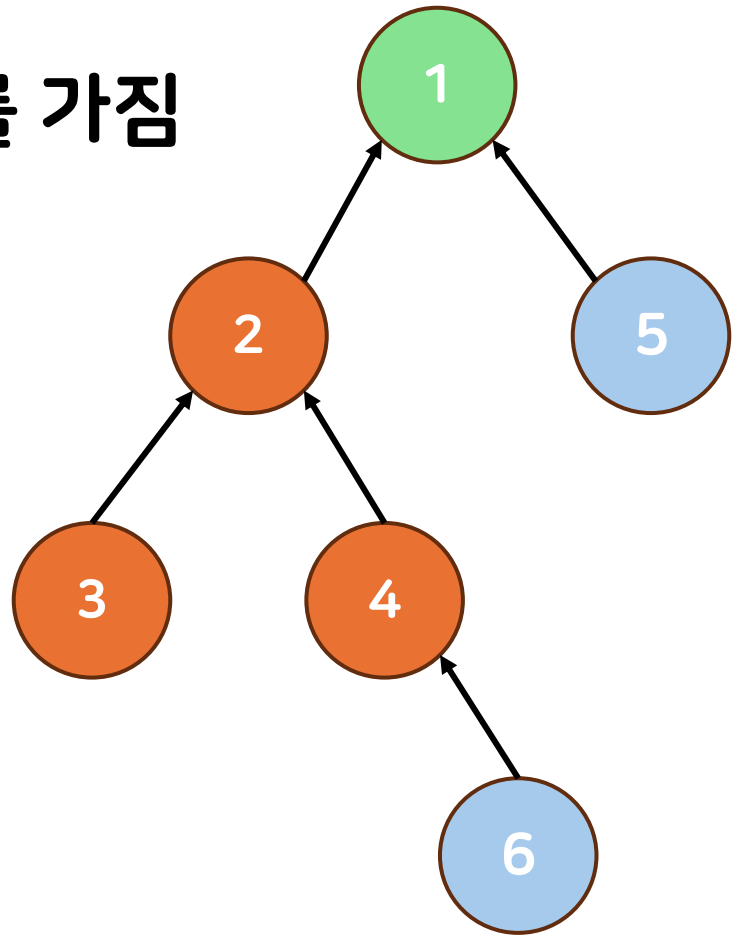


LCA2 / 11438

한번에 여러 번 올라가는 방법은 없나?

루트를 제외한 각 정점은 하나의 부모를 가짐

루트의 부모는 편의상 루트로 두면
각 정점은 outdegree가 1개
-> 함수형 그래프



LCA2 / 11438

lca를 구하는 방법

1. 우선 깊이를 맞춘다
2. 두 정점이 같은 최소의 조상까지 올라간다

1 -> 깊이가 더 깊은 정점을 Sparse Table로 log(N)에 올려주면 됨

```
void build(){
    for(int i = 1;i < 22;i++){
        for(int x = 1;x <= n;x++){
            int mid = p[i - 1][x];
            p[i][x] = p[i - 1][mid];
        }
    }
}
```

```
if(dep[a] > dep[b]) swap(a, b);
int diff = dep[b] - dep[a];
for(int i = 0;i < 22;i++){
    if((diff & (1 << i)) == 0) continue;
    b = p[i][b];
}

if(a == b) return a;
```

LCA2 / 11438

lca를 구하는 방법

1. 우선 깊이를 맞춘다
2. 두 정점이 같은 최소의 조상까지 올라간다

2 -> 2^i 번째 조상을 봤을 때 두 정점이 다를 때만
2^i 번째 조상으로 올라 감

-> 같은 정점이 되지 않는 조상 중
가장 깊이가 낮은 조상으로 올라 감
마지막으로 부모를 반환하면 됨

```
for(int i = 21;i >= 0;i--){
    if(p[i][a] == p[i][b]) continue;
    a = p[i][a]; b = p[i][b];
}

return p[0][a];
```

LCA2 / 11438

시간 복잡도

DFS -> O(N)

build() -> O(N log N)

lca() -> O(log N)

O((N + M) log N)

```cpp
vector <int> adj[MAX];
int p[22][MAX], dep[MAX];
void dfs(int cur, int pre){
    p[0][cur] = pre;
    dep[cur] = dep[pre] + 1;
    for(auto& nxt : adj[cur]){
        if(pre == nxt) continue;
        dfs(nxt, cur);
    }
}
```

```cpp
void run(){
    cin >> n;
    for(int i = 1;i < n;i++){
        int s, e; cin >> s >> e;
        adj[s].push_back(e);
        adj[e].push_back(s);
    }

    dfs(1, 1); build();
    cin >> m;
    while(m--){
        int a, b; cin >> a >> b;
        cout << lca(a, b) << "\n";
    }
}
```

```cpp
void build(){
    for(int i = 1;i < 22;i++){
        for(int x = 1;x <= n;x++){
            int mid = p[i - 1][x];
            p[i][x] = p[i - 1][mid];
        }
    }
}

int lca(int a, int b){
    if(dep[a] > dep[b]) swap(a, b);
    int diff = dep[b] - dep[a];
    for(int i = 0;i < 22;i++){
        if((diff & (1 << i)) == 0) continue;
        b = p[i][b];
    }

    if(a == b) return a;
    for(int i = 21;i >= 0;i--){
        if(p[i][a] == p[i][b]) continue;
        a = p[i][a]; b = p[i][b];
    }

    return p[0][a];
}
```

질문?

고생하셨습니다